



Contents

13.1 Abundance of Digitized Text	574
13.1.1 Notations for Character Strings	575
13.2 Pattern-Matching Algorithms	576
13.2.1 Brute Force	576
13.2.2 The Boyer-Moore Algorithm	578
13.2.3 The Knuth-Morris-Pratt Algorithm	582
13.3 Tries	586
13.3.1 Standard Tries	586
13.3.2 Compressed Tries	590
13.3.3 Suffix Tries	592
13.3.4 Search Engine Indexing	594
13.4 Text Compression and the Greedy Method	595
13.4.1 The Huffman Coding Algorithm	596
13.4.2 The Greedy Method	597
13.5 Dynamic Programming	598
13.5.1 Matrix Chain-Product	598
13.5.2 DNA and Text Sequence Alignment	601
13.6 Exercises	605

13.1 Abundance of Digitized Text

Despite the wealth of multimedia information, text processing remains one of the dominant functions of computers. Computers are used to edit, store, and display documents, and to transport files over the Internet. Furthermore, digital systems are used to archive a wide range of textual information, and new data is being generated at a rapidly increasing pace. A large corpus can readily surpass a petabyte of data (which is equivalent to a thousand terabytes, or a million gigabytes). Common examples of digital collections that include textual information are:

- Snapshots of the World Wide Web, as Internet document formats HTML and XML are primarily text formats, with added tags for multimedia content
- All documents stored locally on a user's computer
- Email archives
- Compilations of status updates on social networking sites such as Facebook
- Feeds from microblogging sites such as Twitter and Tumblr

These collections include written text from hundreds of international languages. Furthermore, there are large data sets (such as DNA) that can be viewed computationally as “strings” even though they are not language.

In this chapter, we explore some of the fundamental algorithms that can be used to efficiently analyze and process large textual data sets. In addition to having interesting applications, text-processing algorithms also highlight some important algorithmic design patterns.

We begin by examining the problem of searching for a pattern as a substring of a larger piece of text, for example, when searching for a word in a document. The pattern-matching problem gives rise to the *brute-force method*, which is often inefficient but has wide applicability. We continue by describing more efficient algorithms for solving the pattern-matching problem, and we examine several special-purpose data structures that can be used to better organize textual data in order to support more efficient runtime queries.

Because of the massive size of textual data sets, the issue of compression is important, both in minimizing the number of bits that need to be communicated through a network and to reduce the long-term storage requirements for archives. For text compression, we can apply the *greedy method*, which often allows us to approximate solutions to hard problems, and for some problems (such as in text compression) actually gives rise to optimal algorithms.

Finally, we introduce *dynamic programming*, an algorithmic technique that can be applied in certain settings to solve a problem in polynomial time, which appears at first to require exponential time to solve. We demonstrate the application on this technique to the problem of finding partial matches between strings that may be similar but not perfectly aligned. This problem arises when making suggestions for a misspelled word, or when trying to match related genetic samples.

13.1.1 Notations for Character Strings

When discussing algorithms for text processing, we use character strings as a model for text. Character strings can come from a wide variety of sources, including scientific, linguistic, and Internet applications. Indeed, the following are examples of such strings:

```
S = "CGTAAACTGCTTTAATCAAACGC"
T = "http://www.wiley.com"
```

The first string, S , comes from DNA applications, and the second string, T , is the Internet address (URL) for the publisher of this book.

To allow fairly general notions of a string in our algorithm descriptions, we only assume that characters of a string come from a known **alphabet**, which we denote as Σ . For example, in the context of DNA, there are four symbols in the standard alphabet, $\Sigma = \{A, C, G, T\}$. This alphabet Σ can, of course, be a subset of the ASCII or Unicode character sets, but it could also be something more general. Although we assume that an alphabet has a fixed finite size, denoted as $|\Sigma|$, that size can be nontrivial, as with Java's treatment of the Unicode alphabet, which allows more than a million distinct characters. We therefore consider the impact of $|\Sigma|$ in our asymptotic analysis of text-processing algorithms.

Java's `String` class provides support for representing an **immutable** sequence of characters, while its `StringBuilder` class supports **mutable** character sequences (see Section 1.3). For much of this chapter, we rely on the more primitive representation of a string as a **char** array, primarily because it allows us to use the standard indexing notation $S[i]$, rather than the `String` class's more cumbersome syntax, `S.charAt(i)`.

In order to discuss pieces of a string, we denote as a **substring** of an n -character string P a string of the form $P[i]P[i+1]P[i+2]\cdots P[j]$, for some $0 \leq i \leq j \leq n-1$. To simplify the notation for referring to such substrings in prose, we let $P[i..j]$ denote the substring of P from index i to index j inclusive. We note that string is technically a substring of itself (taking $i = 0$ and $j = n-1$), so if we want to rule this out as a possibility, we must restrict the definition to **proper** substrings, which require that either $i > 0$ or $j < n-1$. We use the convention that if $i > j$, then $P[i..j]$ is equal to the **null string**, which has length 0.

In addition, in order to distinguish some special kinds of substrings, let us refer to any substring of the form $P[0..j]$, for $0 \leq j \leq n-1$, as a **prefix** of P , and any substring of the form $P[i..n-1]$, for $0 \leq i \leq n-1$, as a **suffix** of P . For example, if we again take P to be the string of DNA given above, then "CGTAA" is a prefix of P , "CGC" is a suffix of P , and "TTAATC" is a (proper) substring of P . Note that the null string is a prefix and a suffix of any other string.